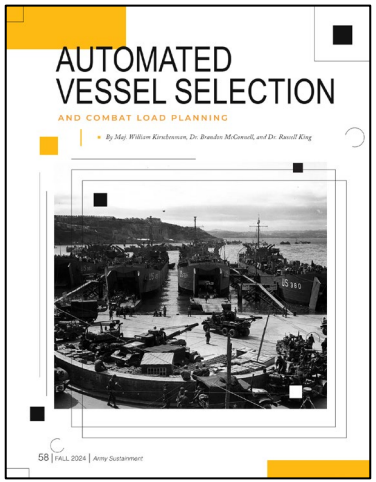
Automating Combat Load Planning using a Prioritized 2-D Packing Approach

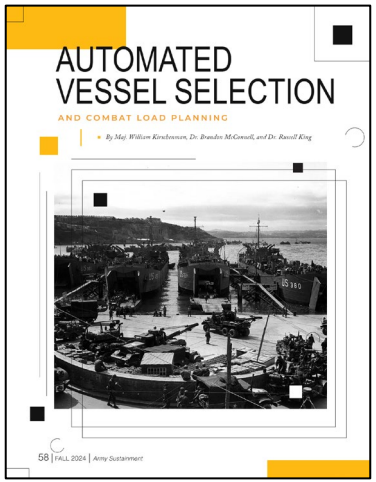


63rd Army Operations Research Symposium
WG 4: Sustainment
September 23, 2025

MAJ Will Kirschenman

Operations Research Ph.D. Candidate
North Carolina State University

Purpose: Introduce a novel optimization framework that automates and enhances military combat load planning by systematically integrating multi-level tactical priorities and critical operational constraints, addressing gaps in current methodologies.

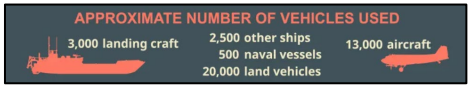


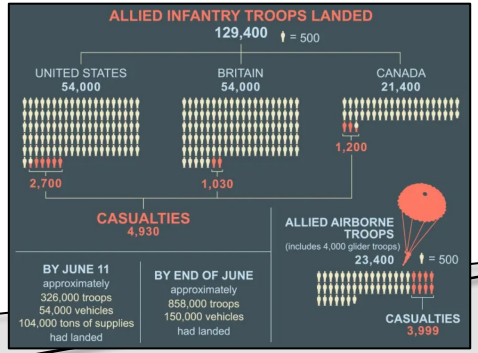
Agenda:

- Introduction.
- Baseline Model.
- Literature Review and Model Enhancements.
- Solution Techniques.
- Experimentation and Results.
- Conclusion.

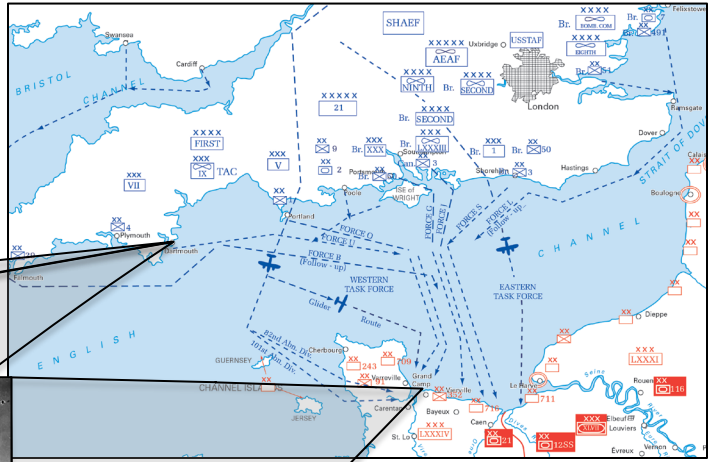
Normandy Invasion: June 6, 1944

- **Initial detailed planning** started in March 1943.
- **Combat loading** commenced in April 1944.
- Over 1,200 landing craft were loaded in 14 ports across southern England, each meticulously scheduled to avoid congestion on June 6, 1944.





Allied Invasion Force and German Dispositions, Northern France, 1944.



1ID Artillery Equipment Loading on LSTs at Brixham, England, on June 1, 1944.

Off-loading equipment on Omaha Beach in days following June 6, 1944.

Other Examples

- Incheon Landing.
- Operation Cartwheel (Rabaul).
- Operation Husky (Invasion of Sicily).
- Operation Iceberg (Battle of Okinawa).

What if we need to do this again?

Motivation

- **Tactical Sequencing (Order of March).** Equipment must be accessible and off-loading in the specific sequence required by the ground tactical plan.
- **Unit Integrity / Group Cohesion.** Military units are designed to fight as cohesive elements.
- **Asset Prioritization.** Certain assets may have higher intrinsic priority based on role and require preferential placement.
- **Speed of Off-Load.** In contested environments, the time spent off-loading is a period of high vulnerability.

1ID Artillery Equipment Loading on LSTs at Brixham, England, on June 1, 1944.

Off-loading equipment on Omaha Beach in days following June 6, 1944.

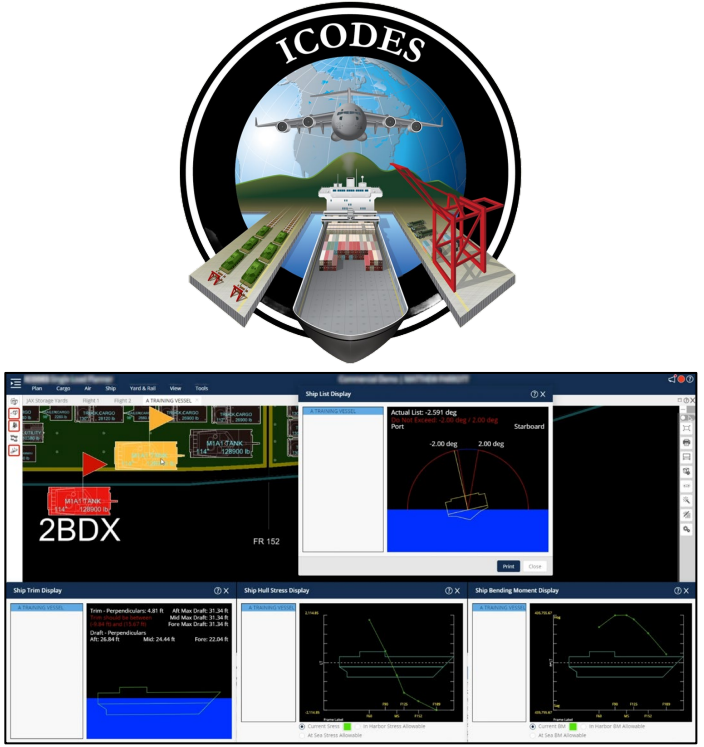
Introduction

Combat Loading Context:

- **Application:** Prioritized loading of a single vessel.
- **Goal:** Enable rapid, effective off-loading in contested environments.

Role of ICODES (Integrated Computerized Deployment System):

- **Purpose:** DoD standard system for load planning (Sea, Air, Land). Improves efficiency, safety, visibility.
- **Architecture:** Ontology-based, agent-driven system interconnecting data.
  - Uses shared data libraries (equipment, vessels).
  - Software "agents" collaborate (Stow, Stability, Hazards, etc.).
- **Key Functions:** Performs automated feasibility checks:
  - Vessel Trim & Stability.
  - HAZMAT Segregation.
  - Cargo Accessibility and Clearances.
- **Limitations:** Achieving *tactical priorities* often requires manual adjustments ("User Stow") beyond automated checks/basic "Auto Stow."



ICODES Single Load Planner module.

Methodology

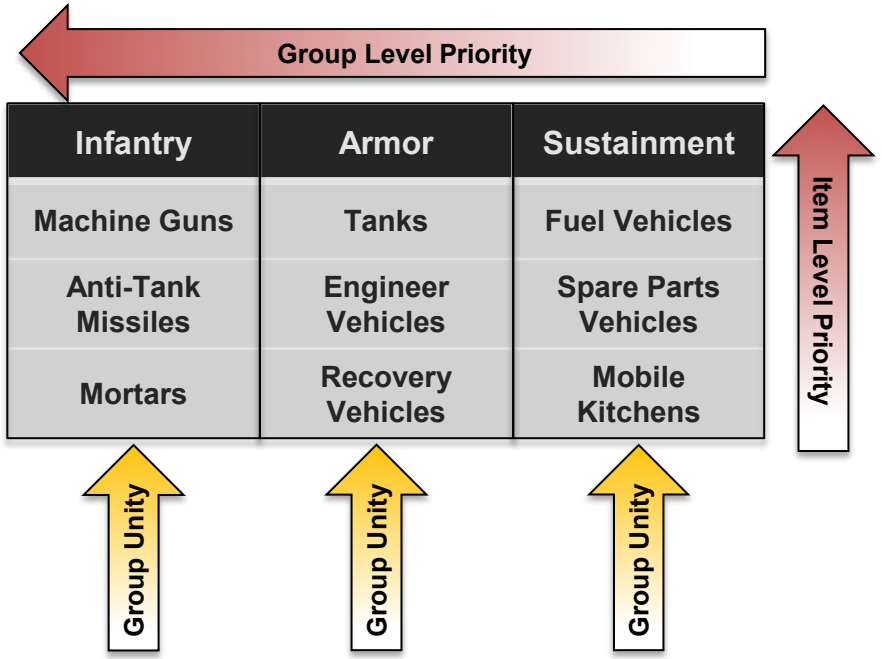
Formulation

Approach: Mixed-Integer Linear Program (MILP) based on efficient BPP formulations (Fontaine and Minner, 2023).

- Use a **Prioritization Matrix ($\pi_{i,k}$)** for spatial preferences:
  - **Diagonal ($\pi_{i,k}$ for $i = k$):** Priority weight for placing item *i* close to the bin access point (x^o, y^o). *Higher value = stronger pull to access point.*
  - **Off-Diagonal ($\pi_{i,k}$ for $i < k$):** Priority weight for placing items i and k (often same group) close to each other (cohesion). *Higher value = stronger pull together.*
- Optimize item placement by minimizing a **Weighted-Distance Objective Function** (rectilinear distance):

$$\min \sum_{i \in I} \sum_{k \in I, i \leq k} \pi_{ik} \left(\rho_{ik}^{x} + \rho_{ik}^{y} \right)$$

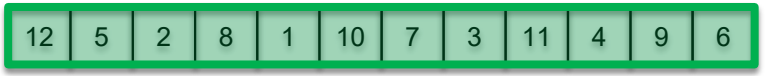
- **Key Constraint Categories:**
  - (1) Ensuring items fit within bin boundaries and reflect chosen orientation. (2) Preventing overlap between any pair of items. (3) Calculating rectilinear distance between item centroids and/or access point.



Solution Techniques

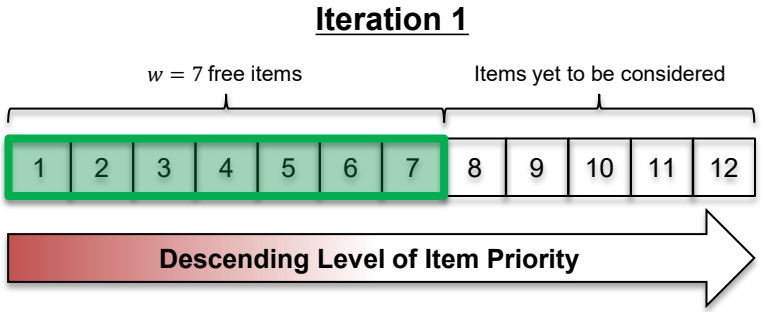
Solution Techniques

1. **Single MILP Approach:**
   - Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
   - Can prove optimality, but computationally expensive (often intractable for > ~7 items within reasonable time).



---

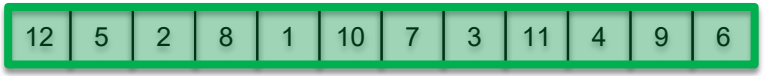
2. **Sliding-Window Matheuristic:**
   - Break the problem into smaller, manageable subproblems based on item priority ranking.
   - Iteratively solve MILPs for a small 'window' (w) of items.
   - Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
   - *Benefit:* Avoids solving large MILP, significantly faster.
   - Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

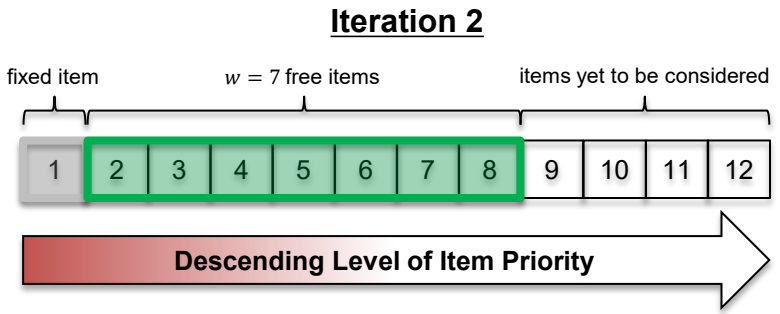
Solution Techniques

1. **Single MILP Approach:**
   - Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
   - Can prove optimality, but computationally expensive (often intractable for > ~7 items within reasonable time).



---

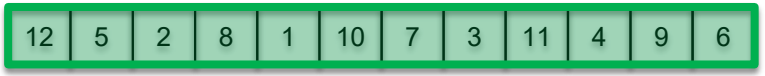
2. **Sliding-Window Matheuristic:**
   - Break the problem into smaller, manageable subproblems based on item priority ranking.
   - Iteratively solve MILPs for a small 'window' (w) of items.
   - Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
   - *Benefit:* Avoids solving large MILP, significantly faster.
   - Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

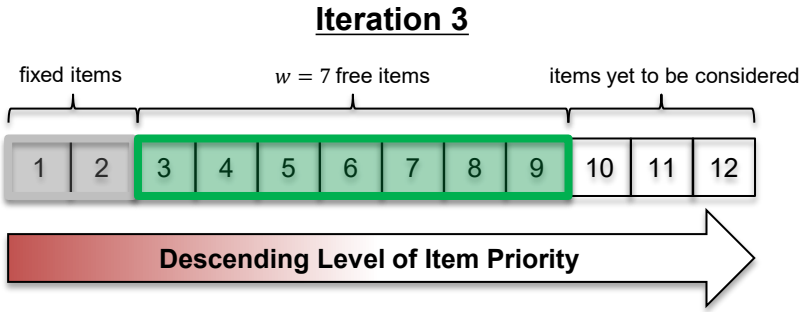
Solution Techniques

1. **Single MILP Approach:**
   - Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
   - Can prove optimality, but computationally expensive (often intractable for > ~7 items within reasonable time).



---

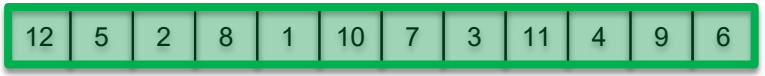
2. **Sliding-Window Matheuristic:**
   - Break the problem into smaller, manageable subproblems based on item priority ranking.
   - Iteratively solve MILPs for a small 'window' (w) of items.
   - Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
   - *Benefit:* Avoids solving large MILP, significantly faster.
   - Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

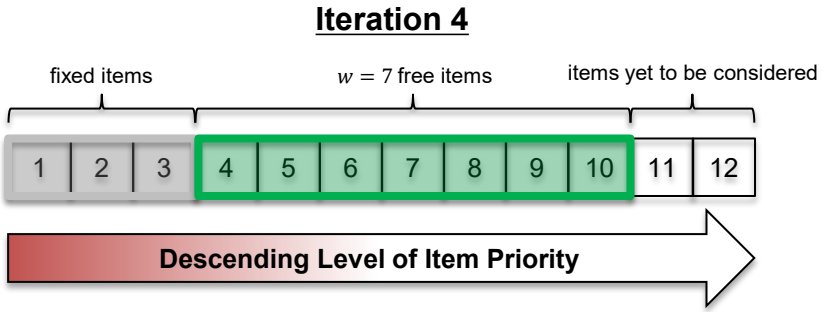
Solution Techniques

1. **Single MILP Approach:**
   - Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
   - Can prove optimality, but computationally expensive (often intractable for > ~7 items within reasonable time).



---

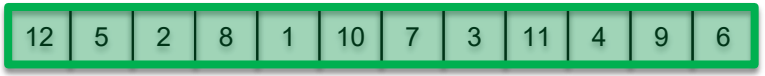
2. **Sliding-Window Matheuristic:**
   - Break the problem into smaller, manageable subproblems based on item priority ranking.
   - Iteratively solve MILPs for a small 'window' (w) of items.
   - Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
   - *Benefit:* Avoids solving large MILP, significantly faster.
   - Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

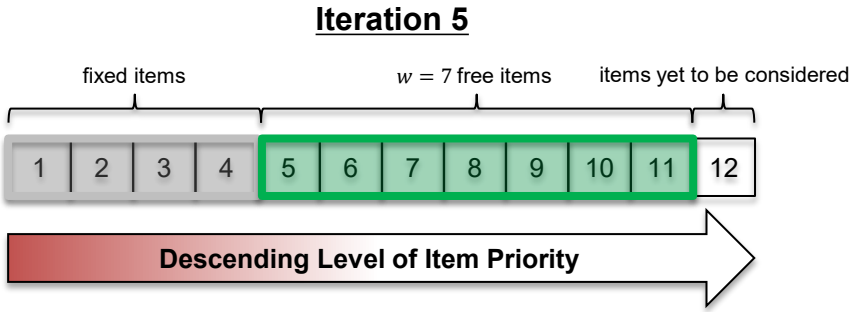
Solution Techniques

1. **Single MILP Approach:**
   - Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
   - Can prove optimality, but computationally expensive (often intractable for > ~7 items within reasonable time).



---

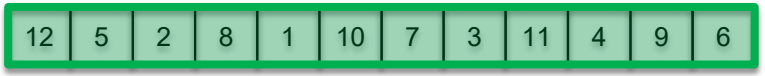
2. **Sliding-Window Matheuristic:**
   - Break the problem into smaller, manageable subproblems based on item priority ranking.
   - Iteratively solve MILPs for a small 'window' (w) of items.
   - Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
   - *Benefit:* Avoids solving large MILP, significantly faster.
   - Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

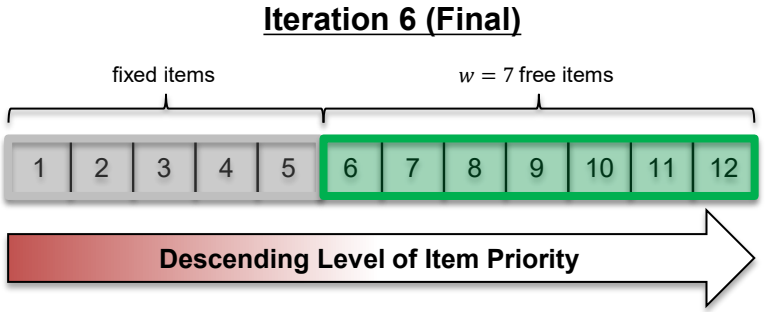
Solution Techniques

1. **Single MILP Approach:**
   - Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
   - Can prove optimality, but computationally expensive (often intractable for > ~7 items within reasonable time).



---

2. **Sliding-Window Matheuristic:**
   - Break the problem into smaller, manageable subproblems based on item priority ranking.
   - Iteratively solve MILPs for a small 'window' (w) of items.
   - Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
   - *Benefit:* Avoids solving large MILP, significantly faster.
   - Parameter w (window size) allows trade-off between computation time and solution quality.



Literature Review

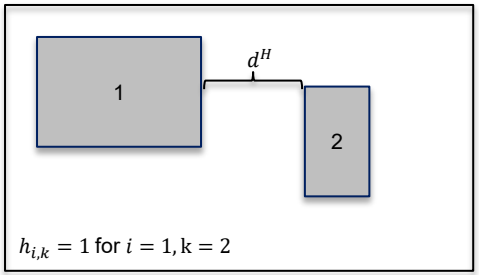
- **Combat Loading Doctrine & Planning Tools.**
  - **Doctrine:** Military combat loading prioritizes tactical needs – rapid offload sequence and unit cohesion – over maximizing cargo density, a principle validated by historical large-scale operations.
  - **Planning Tools (ICODES):** The DoD's standard system, ICODES, excels at feasibility analysis (e.g., stability, safety checks) but lacks automated tactical optimization, often requiring extensive manual adjustments by planners to meet a commander's intent.
  - **The Research Gap:** There is a clear need for an optimization engine to complement ICODES by automatically generating high-quality, tactically-sound initial layouts, thereby reducing planner burden and improving the planning process.
- **Packing with Spatial Preferences.**
  - **Foundation:** Our work builds upon two established fields: *2D Bin Packing*, for its geometric considerations, and *Facility Layout Planning*, for its use of distance-based objectives.
  - **Prioritized Packing Model:** We *extend our prior work* which introduced a novel objective function that integrates item-to-access-point proximity (for off-load speed) and item-to-item cohesion (for unit integrity) into a single, unified packing formulation.
- **Operational Constraints in Packing Literature.**
  - **Material Separation:** Modeled via minimum edge distances; analogous to HAZMAT rules in safe process plant layouts (Patsiatzis et al., 2004).
  - **Obstacle Avoidance:** Modeled as fixed items using standard non-overlap logic (Chircop & Surendonk, 2019).
  - **Load Balancing:** Critical for stability; modeled via center of gravity constraints (Trivella & Pisinger, 2016; Ramos et al., 2018; Davies & Bischoff, 1999).

Model Enhancements

Material Separation

- **Purpose:** Maintain minimum edge-to-edge distance between specific item pairs (e.g., HAZMAT).
- **Approach:** Modify standard non-overlap constraints to add required separation gap.
- **Key Elements:**
  - Set of pairs requiring separation (H).
  - Minimum distance parameter (d_H), and binary indicator for pairs ($h_{i,k}$).
- **Constraints:** Integrate separation distance d_H into non-overlap logic when $h_{i,k} = 1$.

$$x_k^r \le x_i - (h_{i,k} \cdot d^H) + (1 - a_{i,k}^x)L \quad \forall i,k \in I^E,\ i < k$$
$$x_i^r \le x_k - (h_{i,k} \cdot d^H) + (1 - a_{k,i}^x)L \quad \forall i,k \in I^E,\ i < k$$
$$y_k^r \le y_i - (h_{i,k} \cdot d^H) + (1 - a_{i,k}^y)W \quad \forall i,k \in I^E,\ i < k$$
$$y_i^r \le y_k - (h_{i,k} \cdot d^H) + (1 - a_{k,i}^y)W \quad \forall i,k \in I^E,\ i < k$$



Example showing minimum material separation distance between a pair of items.

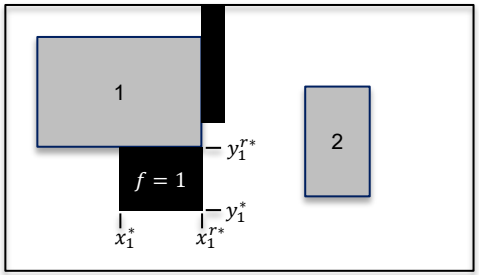
Takeaway: *This approach integrates material separation directly into the existing non-overlap logic without requiring new constraint types.*

Model Enhancements

Obstacle Avoidance

- **Purpose:** Prevent cargo overlap with fixed deck obstacles (pillars, etc.).
- **Approach:** Model obstacles as pre-positioned, immovable "items."
- **Key Elements:**
  - Set of obstacle items (I^F).
  - Known, fixed coordinates for each obstacle: $x_f^*, y_f^*, x_f^{r*}, y_f^{r*}, \forall f \in I^F$.
- **Constraints:** Equality constraints fix obstacle positions:

$$x_f = x_f^* \quad \forall f \in I^F$$
$$y_f = y_f^* \quad \forall f \in I^F$$
$$x_f^r = x_f^{r*} \quad \forall f \in I^F$$
$$y_f^r = y_f^{r*} \quad \forall f \in I^F$$



Example showing obstacles (black fill) treated as fixed items and moveable items (gray fill) as "free" items.

Takeaway: *This approach integrates obstacle handling directly into the existing non-overlap logic without requiring new constraint types.*

Model Enhancements

Load Balancing

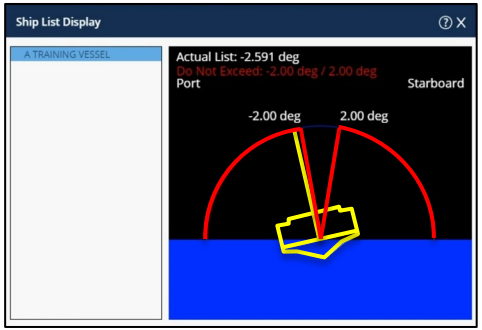
- **Purpose:** Ensure cargo Center of Gravity (CG) is within safe limits for vessel stability.
- **Approach:** Add constraints restricting the calculated overall CG of placed cargo items (I^E).
- **Key Elements:**
  - Item weights (m^i), total cargo weight ($M^E = \sum_{i \in I^E} m^i$), target CG location ($B^x_{opt}, B^y_{opt}$), and tolerances ($\delta_x, \delta_y$).
  - Variables for calculated CG (B^x, B^y).
- **Constraints:** Calculate actual cargo CG and enforce within tolerance box:

$$\sum_{i \in I^E} m_i \left(\frac{x_i + x_i^r}{2} \right) = M^E \cdot B^x$$

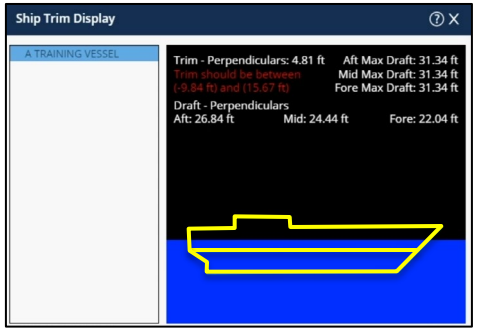
$$\sum_{i \in I^E} m_i \left(\frac{y_i + y_i^r}{2} \right) = M^E \cdot B^y$$

$$B^x_{\text{opt}} - \delta_x \leq B^x \leq B^x_{\text{opt}} + \delta_x$$

$$B^y_{\text{opt}} - \delta_y \leq B^y \leq B^y_{\text{opt}} + \delta_y$$



ICODES Single Load Planner: Ship list agent notifying the user of list (starboard-to-port) stability violations.



ICODES Single Load Planner: Ship trim agent notifying the user of trim (bow-to-stern) stability violations.

Takeaway: *These constraints ensure the center of gravity of the placed cargo items falls within the desired tolerance box.*

Solution Techniques

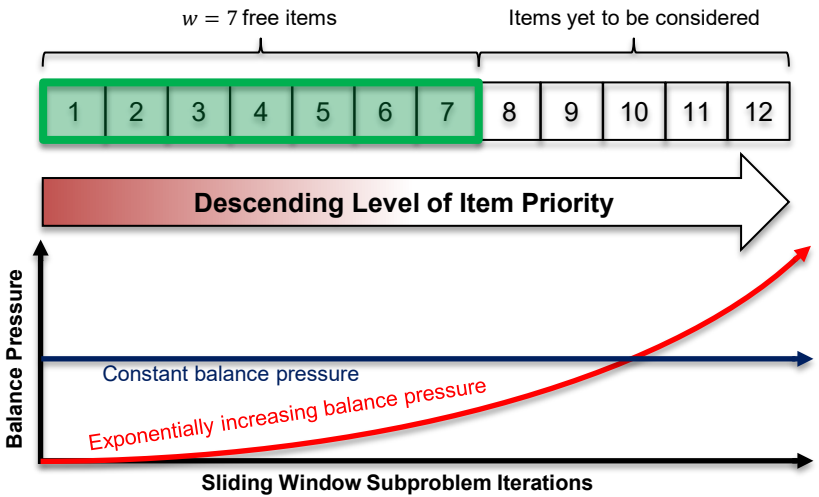
Challenge: Global Constraints in Local Decomposition

Challenge: Load balancing is a global constraint (depends on ALL items), but the sliding-window matheuristic solves local subproblems (window w).

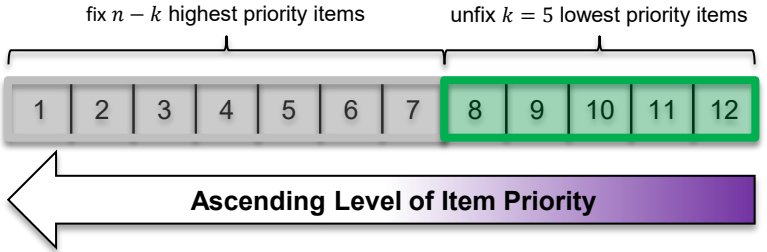
Baseline: Single MILP (with load balancing) works but has lower prioritization scores (previous work).

Goal: Adapt effective sliding-window matheuristic method for load balancing.

In-Stride Balancing



Post-Processing

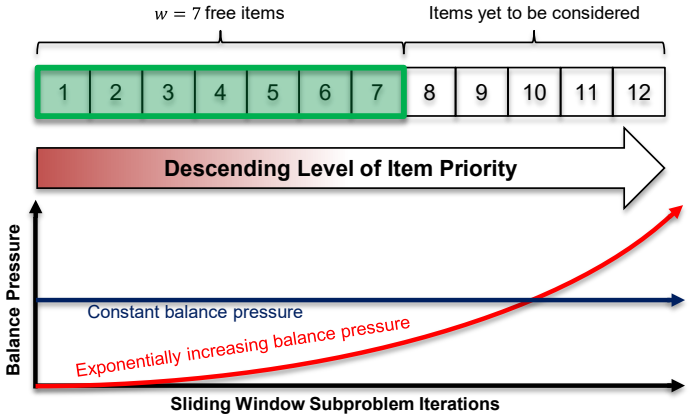


Solution Techniques

In-Stride Load Balancing

Strategy 1: In-Stride Balancing (Penalized Objective):

- **Concept:** Integrate balancing *during* sliding window iterations.
- **Methods:**
  - Constant λ: Fixed penalty on CG deviation.
  - Dynamic λ: Adaptive penalty (e.g., “cold-to-hot” schedule).
  - In-Out: Additional scaled penalty to always pull the cargo items’ CG toward the optimal bin CG.
- **Penalty Scaling:** Custom grid-based lower bound (GB-LB) used to guide λ magnitude, ensuring consistent trade-off (GB-LB was developed in previous work).



Solution Techniques

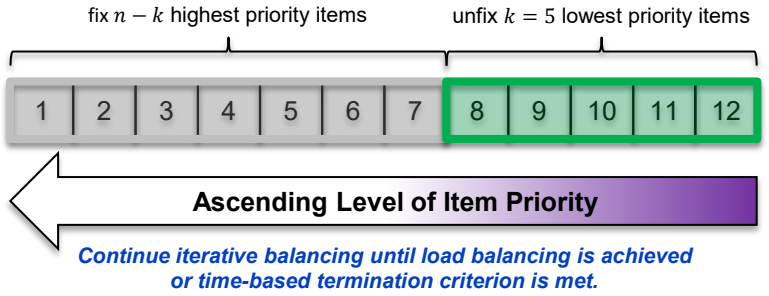
Post-Processing Adjustment for Load Balancing

Strategy 2: Post-Processing Adjustment (Two-Stage):

- **Stage 1: Any Matheuristic Solve that is Not Both *Feasible* and *Load Balanced*:**
  - Run standard sliding-window matheuristic or in-stride balancing variant.
  - Result: A layout that might be infeasibly packed and/or unbalanced.
- **Stage 2: Iterative Feasibility-Seeking MILP Adjustment:**
  - **Guiding Principle:** *Systematically increase solution flexibility* from the Stage 1 layout until load-balancing feasibility is achieved. Start with most items fixed, then iteratively unfix coordinates of lowest-priority items (reverse π_{ii} order) and re-optimize.
  - **Unfixing Sequence Techniques:** Reverse priority, x^r-based methods (farthest from off-loading point), greedy CG Impact.
  - **Goal:** Achieve load balance feasibility while *minimally degrading* prioritization score.

Stage 2 Algorithm: Start with Stage 1 solution (coordinates used for warm start + fixing). Set $k = 1$ (number of free items). While load-balancing is not feasible & k is less than the total number of items:

1. **Identify:** Determine the set of k lowest-priority items.
2. **Define MILP:** Full MILP with load-balancing constraints. *Fix* coordinates for all items *not* in the lowest-k set.
3. **Solve MILP:** Use warm start. Apply per-iteration time limit. **Check:** If the final solution from Step 3 satisfies load-balancing constraints → STOP.
4. **Increment:** If not feasible, $k = k + 1$, repeat loop.



Solution Techniques

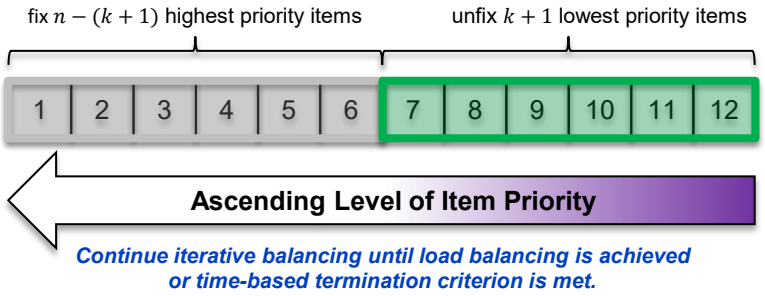
Post-Processing Adjustment for Load Balancing

Strategy 2: Post-Processing Adjustment (Two-Stage):

- **Stage 1: Any Matheuristic Solve that is Not Both *Feasible* and *Load Balanced*:**
  - Run standard sliding-window matheuristic or in-stride balancing variant.
  - Result: A layout that might be infeasibly packed and/or unbalanced.
- **Stage 2: Iterative Feasibility-Seeking MILP Adjustment:**
  - **Guiding Principle:** *Systematically increase solution flexibility* from the Stage 1 layout until load-balancing feasibility is achieved. Start with most items fixed, then iteratively unfix coordinates of lowest-priority items (reverse π_{ii} order) and re-optimize.
  - **Unfixing Sequence Techniques:** Reverse priority, x^r-based methods (farthest from off-loading point), greedy CG Impact.
  - **Goal:** Achieve load balance feasibility while *minimally degrading* prioritization score.

Stage 2 Algorithm: Start with Stage 1 solution (coordinates used for warm start + fixing). Set $k = 1$ (number of free items). While load-balancing is not feasible & k is less than the total number of items:

1. **Identify:** Determine the set of k lowest-priority items.
2. **Define MILP:** Full MILP with load-balancing constraints. *Fix* coordinates for all items *not* in the lowest-k set.
3. **Solve MILP:** Use warm start. Apply per-iteration time limit. **Check:** If the final solution from Step 3 satisfies load-balancing constraints → STOP.
4. **Increment:** If not feasible, $k = k + 1$, repeat loop.



Solution Techniques

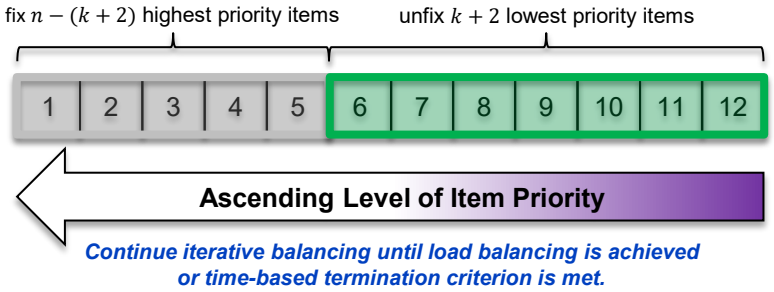
Post-Processing Adjustment for Load Balancing

Strategy 2: Post-Processing Adjustment (Two-Stage):

- **Stage 1: Any Matheuristic Solve that is Not Both *Feasible* and *Load Balanced*:**
  - Run standard sliding-window matheuristic or in-stride balancing variant.
  - Result: A layout that might be infeasibly packed and/or unbalanced.
- **Stage 2: Iterative Feasibility-Seeking MILP Adjustment:**
  - **Guiding Principle:** *Systematically increase solution flexibility* from the Stage 1 layout until load-balancing feasibility is achieved. Start with most items fixed, then iteratively unfix coordinates of lowest-priority items (reverse π_{ii} order) and re-optimize.
  - **Unfixing Sequence Techniques:** Reverse priority, x^r-based methods (farthest from off-loading point), greedy CG Impact.
  - **Goal:** Achieve load balance feasibility while *minimally degrading* prioritization score.

Stage 2 Algorithm: Start with Stage 1 solution (coordinates used for warm start + fixing). Set $k = 1$ (number of free items). While load-balancing is not feasible & k is less than the total number of items:

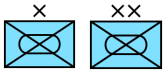
1. **Identify:** Determine the set of k lowest-priority items.
2. **Define MILP:** Full MILP with load-balancing constraints. *Fix* coordinates for all items *not* in the lowest-k set.
3. **Solve MILP:** Use warm start. Apply per-iteration time limit. **Check:** If the final solution from Step 3 satisfies load-balancing constraints → STOP.
4. **Increment:** If not feasible, $k = k + 1$, repeat loop.



Data Generation Strategy

- **Goal:** Create realistic, reproducible military scenario instances.
- **Data Sources:**

JECD (Joint Equipment Characteristic Database): Item dimensions, weight.



TOE (Table of Organizational Equipment) / MTOE (Modified TOE): Unit structures (e.g., Armored Brigade), item quantities).

- **Filtering:** Use EQUIP CODEs to select relevant combat/support vehicles.
- **Groups and Priorities:** Assign items to functional groups (UIC-based); assign plausible tactical priorities.
- **Vessel Specifications:** Define representative Navy/MSC vessel decks (dimensions, target CG, balance tolerance).
- **Instance Generation:** Assign random subsets of groups (ABCT to Division scale) to vessels:
  - consider packing density range: 65%-85% in increments of 5%.
  - 70 total instances with an even representation of vessel types.
  - Four CG deviation tolerances: 1%, 5%, 10%, 15%.

EQUIPMENT CODE (EQUIP CODE): An alphanumeric character code applied to each vehicle and equipment data listing to enable automated identification and retrieval of selected data categories. Types of equipment codes and their meanings are as follows:

TYPE EQUIP CODE	MEANING
	Vehicles, wheeled (self-propelled)
1	1/2-ton or less
2	Sedan
3	2-1/2-ton or less
4	Greater than 2-1/2-ton
5	Materials handling equipment (MHE)
8	Construction equipment
	Vehicles, wheeled (not self-propelled)
6	2-1/2-ton or less
7	Greater than 2-1/2-ton
9	Construction equipment
0	Materials handling equipment (MHE)
	Other vehicles
C	Vehicles, tracked or half-tracked (except D and E type equipment codes)
D	Tanks (combat)
E	Artillery (self-propelled)
F	Artillery (towed)
G	Floating craft (except amphibious vehicles that are included under the appropriate numeric code)
H	Aircraft, rotary wing (operational)
J	Aircraft, fixed wing (operational)
K	All aircraft in a reduced configuration
	Nonvehicular and special-handling equipment
M	Class A Explosives
N	Class B Explosives
P	Hazardous items
Q	Ammunition and explosives over .60 caliber
R	Small-arms ammunition .60 caliber or less
S	Yellow TAT organizational equipment and supplies
T	Red TAT organizational equipment and supplies
U	Equipment other than vehicles
V	LVAD (NOTE: Equipment or supplies rigged or packaged for airdrop must be reported as "Special Handling Cargo")
X	Red TAT baggage
Y	Yellow TAT personal baggage

Equipment categorization based on EQUIP CODE used for filtering JECD data.

Key Findings

1. Reliability: Matheuristic Methods Outperform Standard MILP

The standard monolithic *MILP approach is unreliable* for solving the prioritized, load-balanced packing problem, succeeding in only **60.0% of test cases.**

In contrast, our proposed *Sliding-Window (SW) Matheuristic methods are exceptionally reliable*, achieving load balance in **97.5% (SW)** and **98.9% (SW In-Stride)** of instances**.** The remaining SW unbalanced solutions could be balanced with minimal ballast adjustments.

The high success of the SW method is driven by its *robust post-processing stage*, which successfully recovers a balanced solution in nearly all cases where the initial, priority-focused stage does not.

Table 3: Overall load balancing success rates by solution method.

Method	Stage 1 Success	Overall Success	Success Rate
Single MILP	168/280	168/280	60.0%
SW Iterative	140/280	273/280	97.5%
SW In-Stride	263/280	277/280	98.9%

Takeaway: *The low reliability of monolithic MILP formulations necessitates a decomposed matheuristic approach for practical application.*

Key Findings

2. Custom Lower Bounds Provide Superior Quality Insights

Standard Solver Bounds (Gurobi): Standard solver optimality gaps are often *very poor for complex instances*, making it difficult to assess true solution quality.

Grid-Based Lower Bound (GB-LB): Our custom deterministic bound, which leverages the problem's geometric and priority structure, provides a *consistently tighter quality measure* than the solver's bound.

Statistical Lower Bound using Extreme Value Theory: For the largest instances, statistical analysis (Wilson et al, 2004) further *suggests the true optimum is tighter than indicated by deterministic bounds*, with estimated gaps between 2-6%.

Takeaway: *Custom-developed lower bounds are essential for this problem class, providing a more accurate assessment of solution quality than standard solver outputs and demonstrating the effectiveness of our matheuristic solutions.*

Items	Groups	CG Dev	Obj	LB	GB-LB	LB Gap	GB-LB Gap
111	26	0.01	118.44	71.72	90.69	39.44%	23.43%
111	26	0.05	112.22	61.37	90.69	45.32%	19.19%
111	26	0.10	110.05	49.91	90.69	54.65%	17.59%
111	26	0.15	109.87	39.19	90.69	64.33%	17.46%
110	22	0.01	125.38	65.84	95.04	47.49%	24.20%
110	22	0.05	117.41	57.50	95.04	51.03%	19.05%
110	22	0.10	112.97	48.74	95.04	56.86%	15.87%
110	22	0.15	110.07	41.50	95.04	62.29%	13.66%
94	11	0.01	75.23	37.09	61.10	50.70%	18.78%
94	11	0.05	74.19	33.10	61.10	55.38%	17.64%
94	11	0.10	74.29	28.17	61.10	62.08%	17.75%
94	11	0.15	73.82	23.35	61.10	68.37%	17.23%
84	26	0.01	48.45	23.56	37.76	51.38%	22.08%
84	26	0.05	46.37	19.58	37.76	57.77%	18.57%
84	26	0.10	45.56	16.27	37.76	64.29%	17.12%
84	26	0.15	44.85	13.12	37.76	70.75%	15.82%

Comparison of lower bound quality for the 4 largest problem instances by item count.

Instance	CG Dev	Min	Max	Golden-Alt CI	Opt. Gap
84 items	0.01	4.76	4.88	(4.76, 4.88)	2.46%
84 items	0.05	4.57	4.68	(4.57, 4.67)	2.14%
84 items	0.10	4.42	4.56	(4.42, 4.58)	3.49%
84 items	0.15	4.38	4.54	(4.38, 4.55)	3.74%
82 items	0.01	5.63	5.73	(5.63, 5.74)	1.92%
82 items	0.05	5.36	5.46	(5.36, 5.47)	2.01%
82 items	0.10	5.29	5.43	(5.29, 5.45)	2.94%
82 items	0.15	5.31	5.42	(5.31, 5.45)	2.57%
75 items	0.01	3.93	4.10	(3.93, 4.11)	4.38%
75 items	0.05	3.77	3.91	(3.77, 3.92)	3.83%
75 items	0.10	3.65	3.80	(3.65, 3.83)	4.70%
75 items	0.15	3.66	3.80	(3.66, 3.90)	6.15%

Wilson's Statistical Lower Bound results for instance-CG deviation combinations of three large instances at 75%-85% space utilization.

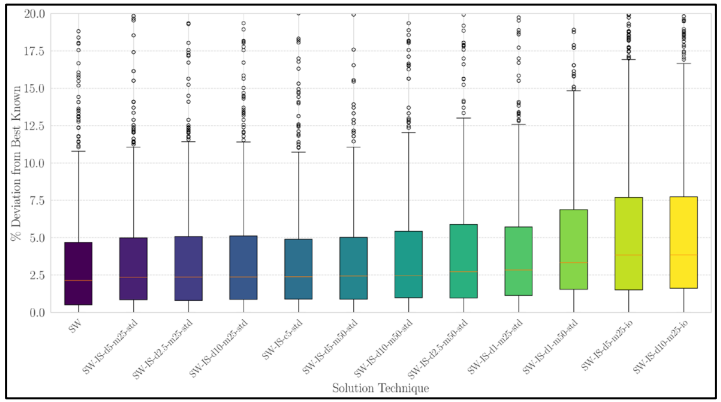
Key Findings

3. The SW Matheuristic Produces Higher-Quality Solutions, Faster

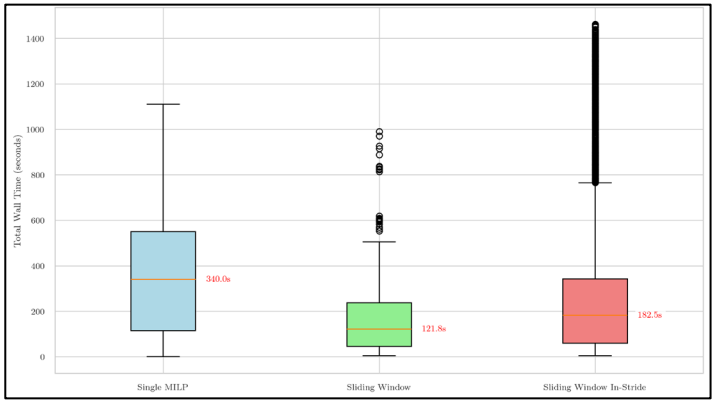
The *SW Matheuristic produces the highest-quality tactical layouts*, achieving an average quality gap of just **4.0%** from the best-known solution.

This significantly outperforms both the Single MILP (**8.9% gap**) and the SW In-Stride Matheuristic (**6.9% gap**), which compromise prioritization quality in an attempt to balance proactively.

The *SW Matheuristic is most computationally efficient*, producing final solutions more than **twice as fast** as the Single MILP.



Distribution of solution quality (percentage deviation from best-known objective) for top-performing techniques.



Wall time (total computational time for the entire process) distribution by solution method.

Takeaway: *A decoupled solution strategy – optimizing for prioritization first, followed by targeted adjustments for load balancing – yields superior tactical layouts with greater computational efficiency.*

Key Findings

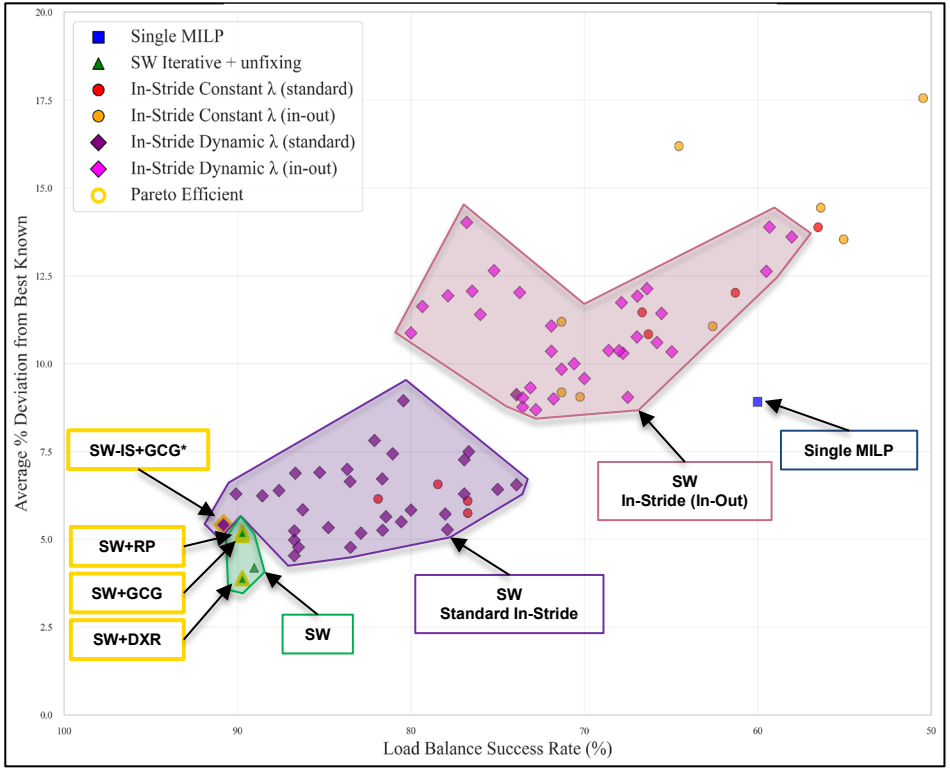
4. Robust Post-Processing is the Key to a Feasible, Load-Balanced Solution

The *core innovation is a universal post-processing framework* that takes a high-quality but potentially infeasible layout and efficiently adjusts it to meet all constraints.

For *maximizing final solution quality*, the **Decreasing x^r (DXR)** unfixing strategy is most effective (3.9% average deviation from best-known objective).

For *maximizing computational efficiency*, the **Greedy CG Impact (GCG)** strategy provides an excellent blend of speed and quality, achieving a high-quality result (5.1% average deviation) in a median of just one iteration.

Takeaway: *The SW Matheuristic's power is its post-processing stage, which reliably transforms infeasible layouts into high-quality, balanced, feasible solutions, ensuring a robust and practical outcome.*



Pareto frontier analysis: load balancing success rate versus solution quality. Points with thicker gold borders represent Pareto efficient configurations.

**This SW-IS+CG configuration had a dynamic lambda schedule with shape-curve value of 10.0 and max λ value of 0.25, creating balance pressure only in the final iteration(s) of the matheuristic procedure.*

Conclusion

Contributions & Future Work

Key Contributions:

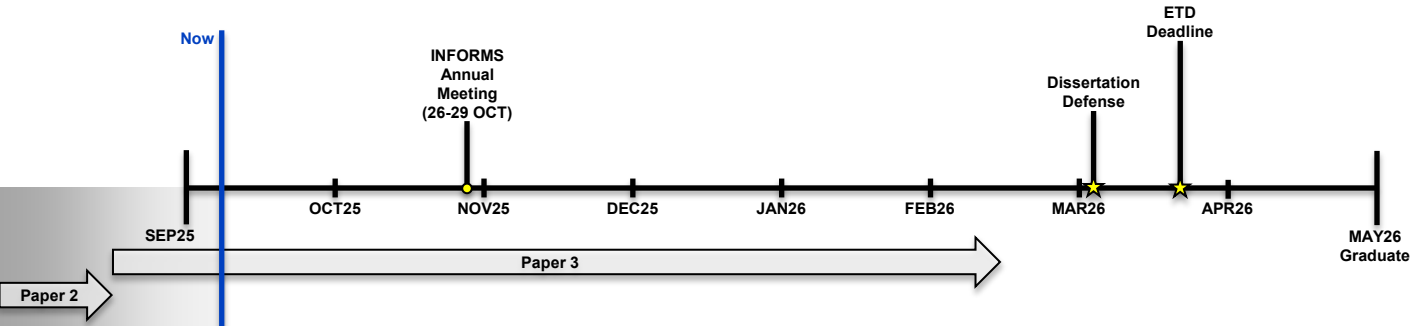
- A *mathematical formulation for combat loading* using prioritized 2D packing that integrates critical operational constraints, such as global load balancing.
- A *robust, multi-stage matheuristic framework* that fundamentally outperforms standard MILP approaches in reliability, solution quality, and computational speed.
- A *method for automatically generating load plans* that are both *operationally feasible* and *optimized for multi-level tactical priorities*.

Limitations:

- Focuses on single-vessel planning and assumes 2D rectangular geometry.
- Matheuristic methods provide high-quality practical solutions but do not guarantee global optimality.

Future Directions & Ongoing Work:

- Develop a *hierarchical optimization framework* for the *multi-vessel, multi-wave* combat loading problem, utilizing the *single-vessel combat loading model* as a *high-fidelity subroutine within a higher-level assignment and scheduling procedure* for large-scale planning.



Will Kirschenman
Operations Research PhD Candidate
North Carolina State University